

EduGenie: AI-Powered Educational Assistant Platform

Project Description:

EduGenie harnesses Generative AI to provide a full-stack educational assistant experience, offering students a fast, personalized, and reliable way to learn. The platform includes an Intelligent Question Answering module that delivers curated academic answers, a Concept Explanation engine that breaks down complex topics with interactive examples, and a Quiz Generator that dynamically creates assessments to test retention on any subject.

Utilizing Fast API with the Google Gemini API, EduGenie processes user inputs, uploaded documents, and learning history to deliver fast, reliable, and intelligent educational assistance. Built with an asynchronous Python backend and a MongoDB Atlas datastore via the Motor async driver, the platform ensures a seamless, secure, and scalable experience. With JWT-based authentication, bcrypt password hashing, and environment-based secret management, EduGenie empowers students, self-learners, and educators to study smarter with confidence.

Scenarios

- Scenario 1: Text Summarization

A student pastes a long academic paragraph or lecture notes into EduGenie's Text Summarization module. The system analyses the input using the Gemini API and generates a concise summary highlighting key concepts and important points. The student then saves the summary into Smart Notes for revision

- Scenario 2: Quiz Generation for Exam Preparation

A student preparing for a Data Structures exam requests EduGenie to generate a 10-question quiz on Binary Search Trees. The Quiz Generator produces multiple-choice questions with varying difficulty levels. After completion, the system evaluates responses and updates the Learning Analytics dashboard with performance insights.

- Scenario 3: Personalized Learning Roadmap

A new learner asks EduGenie to build a structured roadmap for becoming a backend developer. The Adaptive Roadmap engine generates a step-by-step curriculum covering beginner, intermediate, and advanced topics. The learner tracks progress through the Progress Dashboard.

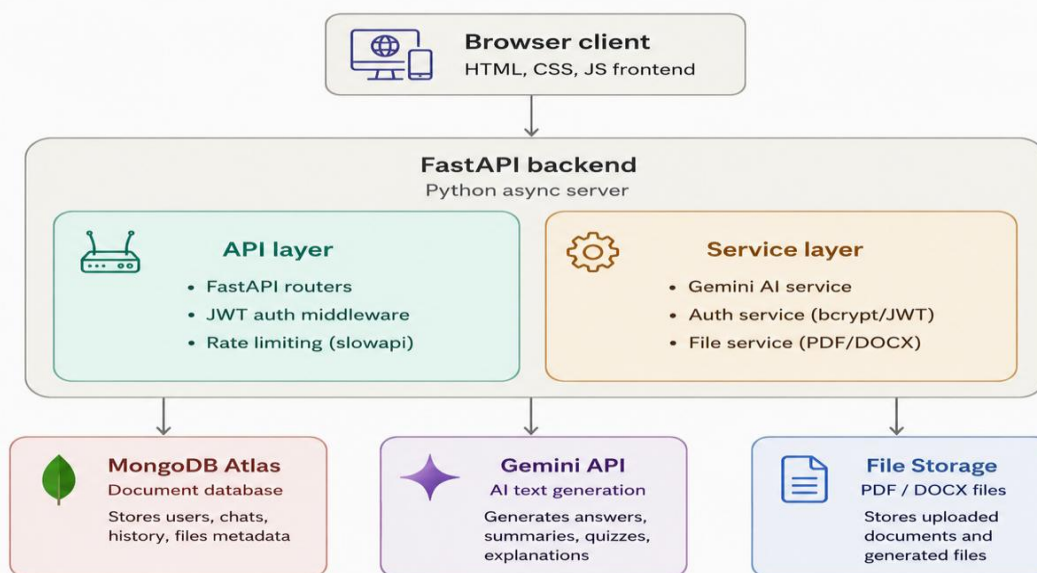
- Scenario 4: Concept Explanation (Doubt Solving)

A student struggling with Operating Systems concepts asks EduGenie to explain "Deadlocks" in simple terms. The system provides a structured explanation with real-world analogies, causes, conditions, and prevention techniques, making the concept easier to understand.

- Scenario 5: Smart Q&A Assistant

A learner preparing for interviews asks EduGenie, "What is normalization in DBMS and why is it important?" The Intelligent Q&A module generates a detailed, structured answer with examples and key points, helping the student quickly revise the topic without searching multiple sources.

Technical Architecture:



Description:

EduGenie follows a modular, layered architecture designed for scalability, maintainability, and high performance. The client interface is built using HTML5, CSS3, and JavaScript, providing a responsive and interactive learning experience. User requests are securely transmitted to the FastAPI backend through RESTful APIs.

The backend manages authentication, request validation, business logic, and AI service orchestration. JWT-based authentication ensures secure access to protected resources, while bcrypt hashing safeguards user credentials. The AI service layer communicates with the Google Gemini API to generate intelligent responses for question answering, concept explanation, quiz generation, text summarization, and personalized learning roadmaps.

MongoDB Atlas is used as the cloud database for storing user profiles, notes, bookmarks, learning history, progress analytics, and application settings. The asynchronous Motor driver enables efficient non-blocking database operations, improving scalability and reducing response time under concurrent user requests. Uploaded files such as PDF, DOCX, and TXT documents are processed through a dedicated file-processing service before being analyzed by the AI engine.

Pre-requisites:

- **Python Programming Proficiency:** Python 3.10+ language features, virtual environments, and async/await syntax.
- **Fast API Framework Knowledge:** Routing, dependency injection, Pydantic models, and middleware.
- **Google Gemini API Familiarity:** Prompting, API key management, and response handling.
- **MongoDB & Motor Async Driver:** Document modelling and asynchronous CRUD operations.
- **HTML, CSS, and JavaScript Skills:** Building responsive, accessible single-page interfaces.
- **JWT Authentication Concepts:** Token issuance, verification, and secure route protection.

Project Workflow:

Milestone 1: Model Selection and System Architecture Design

Activity 1.1: Generate a secure Google Gemini API key and configure it within the project environment using environment variables to ensure safe handling of sensitive credentials and prevent unauthorized access.

Activity 1.2: Conduct detailed research and analysis to identify and select the most suitable generative AI model from Google Gemini that aligns with educational use cases such as question answering, concept understanding, and content generation.

Activity 1.3: Design and define the overall system architecture of the application, clearly illustrating the interaction flow between the frontend interface, backend server, database layer, and external AI services.

Activity 1.4: Set up a robust and scalable development environment by installing and configuring all required libraries, frameworks, and dependencies, including Fast API for backend development, Motor for asynchronous MongoDB operations, and the official Gemini SDK for AI integration.

Milestone 2: Core Functionalities Development

Activity 2.1: Develop and implement the core intelligent features of the platform, including Question Answering, Concept Explanation, Quiz Generation, and Text Summarization modules, ensuring each feature is optimized for educational accuracy and user engagement.

Activity 2.2: Implement the Fast API backend architecture to efficiently manage API routing, handle user requests, process inputs securely, and ensure smooth communication between frontend and AI services.

Milestone 3: Backend (main.py) Development

Activity 3.1: Develop the main application logic within the main.py file by establishing structured API routers for each functional module, integrating authentication mechanisms, and connecting AI response workflows with Google Gemini services.

Milestone 4: Frontend Development

Activity 4.1: Design and develop a modern, responsive, and user-friendly interface using HTML5, CSS3, and JavaScript, ensuring compatibility across different devices and screen sizes.

Activity 4.2: Build dynamic and interactive dashboard components that visually represent AI-generated outputs, user notes, bookmarks, learning progress, and performance analytics in a structured and intuitive format.

Milestone 5: Deployment and System Configuration

Activity 5.1: Prepare the application for local deployment by properly configuring server settings, environment variables, API keys, and required dependencies to ensure smooth execution.

Activity 5.2: Deploy the complete application to a cloud hosting platform such as Render, Railway, or Fly.io, ensuring that the backend services, database connections, and frontend interface are fully integrated and accessible in a production-like environment.

Milestone 6: Project Conclusion and Future Enhancements

Activity 6.1: Perform final system validation and testing to ensure all modules function correctly, including authentication, AI services, and data management components.

Activity 6.2: Document the complete development process, system architecture, API structure, and feature implementations for academic submission and future reference.

Activity 6.3: Identify potential future enhancements such as voice-based learning assistance, AI-powered flashcards, collaborative learning features, multi-language support, mobile application development, and improved caching mechanisms for scalability.

Milestone 1: Model Selection and Architecture

This milestone focuses on selecting the appropriate Google Gemini model and designing the overall architecture of EduGenie. The selected model was evaluated based on its ability to provide accurate question answering, concept explanations, quiz generation, text summarization, and personalized learning support. Along with model selection, the system architecture was designed to define communication between the frontend, Fast API backend, MongoDB Atlas database, and Google Gemini API. The development environment was then configured by installing all required libraries and dependencies, ensuring a secure and scalable foundation for the application.

Activity 1.1: Generate a Gemini API Key

Before the application can communicate with the Gemini LLM, a valid API key is required. Follow the steps below to create one and connect it securely to the project.

- **Step 1 — Create a Google AI Studio Account:** Visit <https://aistudio.google.com> and sign in using a Google account.
- **Step 2 — Navigate to API Keys:** Click “Get API Key” from the left navigation menu, or go directly to the API Keys page.

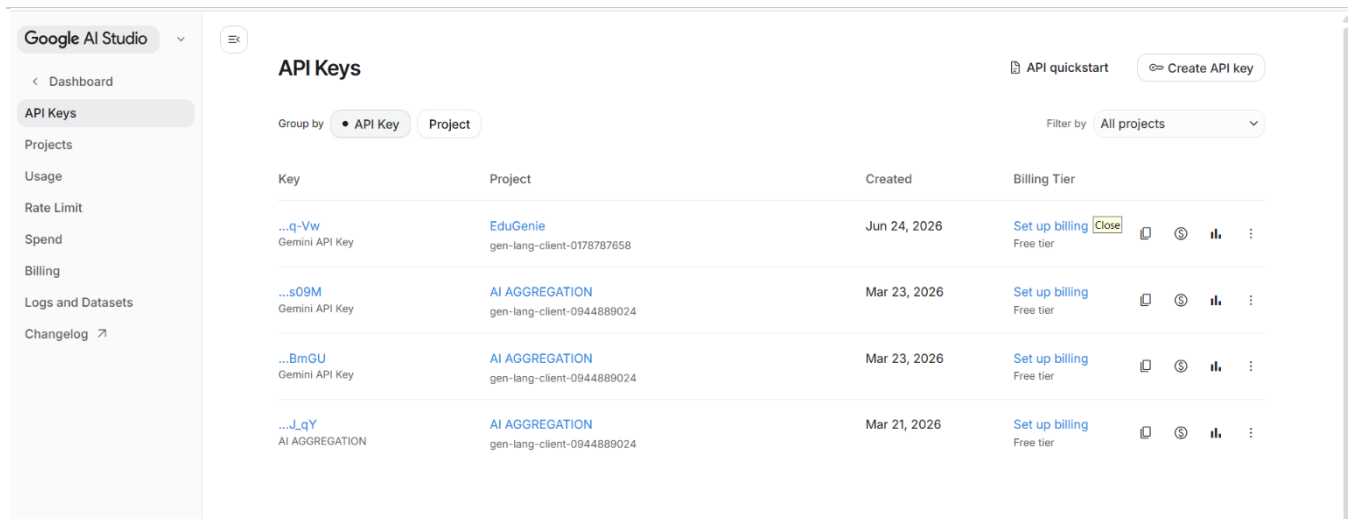


Figure 2 — Google AI Studio API Keys page

- **Step 3 — Create a New API Key:** Click the “Create API Key” button and select the target Google Cloud project.
- **Step 4 — Copy the Key:** The key is displayed once. Copy it immediately and store it in a safe place.
- **Step 5 — Add the Key to Your Project:** Create a .env file in the root of the project directory and paste the key as shown below:

```

5. Project Development Phase > ⚙️ .env
1  # EduGenie Environment Configuration
2  # Copy this file to .env and fill in your values
3
4  # MongoDB Connection String (REQUIRED – paste your connection string here)
5  MONGODB_URI=mongodb+srv://terish5676_db_user:vqww6DwBYkUJ7trL@cluster0.hicacuz.mongodb.net
6  MONGODB_DB_NAME=edugenie
7
8  # JWT Secret Key (REQUIRED – use a strong random string)
9  JWT_SECRET_KEY=DzffJQpzWlVv19IboJwFFvf6jCgEHUssAj40koOK4sw
10 JWT_ALGORITHM=HS256
11 ACCESS_TOKEN_EXPIRE_MINUTES=10080
12
13 # Google Gemini API Key (REQUIRED – get from https://makersuite.google.com/app/apikey)
14 GEMINI_API_KEY=AQ.Ab8RN6KRrq2mToR6QJqAovK7wiwoXbIv7h7b87XtZzTe3UI7Fg
15
16 # App Config
17 APP_NAME=EduGenie
18 APP_VERSION=1.0.0
19 DEBUG=True
20 ALLOWED_ORIGINS=http://localhost:8000,http://127.0.0.1:8000
21
  
```

- **Step 6 — Verify the Key is Loaded:** The config.py file loads this key automatically at startup using python-dotenv:

```

5. Project Development Phase > backend > config.py
1  """
4
5  from pydantic_settings import BaseSettings
6  from typing import List
7
8
9  class Settings(BaseSettings):
10     # App
11     APP_NAME: str = "EduGenie"
12     APP_VERSION: str = "1.0.0"
13     DEBUG: bool = True
14
15     # MongoDB
16     MONGODB_URI: str = "mongodb://localhost:27017"
17     MONGODB_DB_NAME: str = "edugenie"
18
19     # JWT
20     JWT_SECRET_KEY: str = "change-this-secret-key"
21     JWT_ALGORITHM: str = "HS256"
22     ACCESS_TOKEN_EXPIRE_MINUTES: int = 10080 # 7 days
23
24     # Gemini AI
25     GEMINI_API_KEY: str = ""
26

```

- **Important — Never Commit Your API Key:** Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository:

```
.env
```

Activity 1.2: Research and Select the Appropriate Generative AI Model

- **Understand the Project Requirements:** Review the specific needs of EduGenie, focusing on the types of content it will generate (answers, explanations, quizzes, summaries, and roadmaps).
- **Explore Gemini's Model Documentation:** Examine the available Gemini models, including their context windows, speed, and multimodal capabilities.
- **Evaluate Model Performance:** Compare models based on response quality, reasoning depth, and relevance to educational tasks.
- **Select the Optimal Model:** Choose gemini-1.5-flash for its balance of response quality and generation speed for interactive Q&A, with gemini-1.5-pro reserved for longer document summarization tasks.

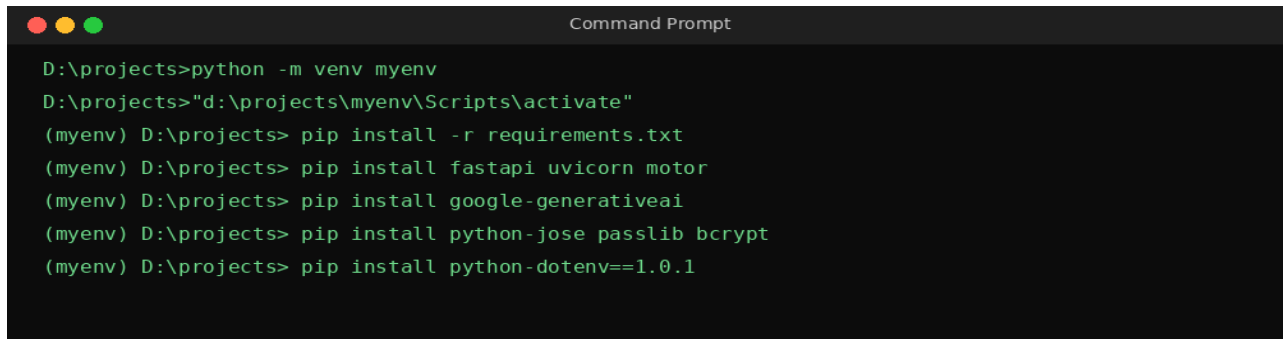
Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including the frontend, Fast API backend, and Gemini API integration.
- **Detail Frontend Functionality:** Outline how users interact with the application — asking questions, uploading documents, generating quizzes, and viewing analytics.
- **Outline Backend Responsibilities:** Specify how the Fast API backend handles incoming requests, validates and sanitizes user input, builds structured prompts, and communicates with the Gemini API.

- Describe AI Integration Points: Define how the backend constructs prompts using the prompts.py utility module, calls the Gemini API through gemini_service.py, and returns structured results to the frontend.

Activity 1.4: Set Up the Development Environment

- Install Python and Pip: Ensure Python 3.10+ is installed along with pip for dependency management.
- Create a Virtual Environment: Set up a virtual environment using venv to isolate project dependencies.

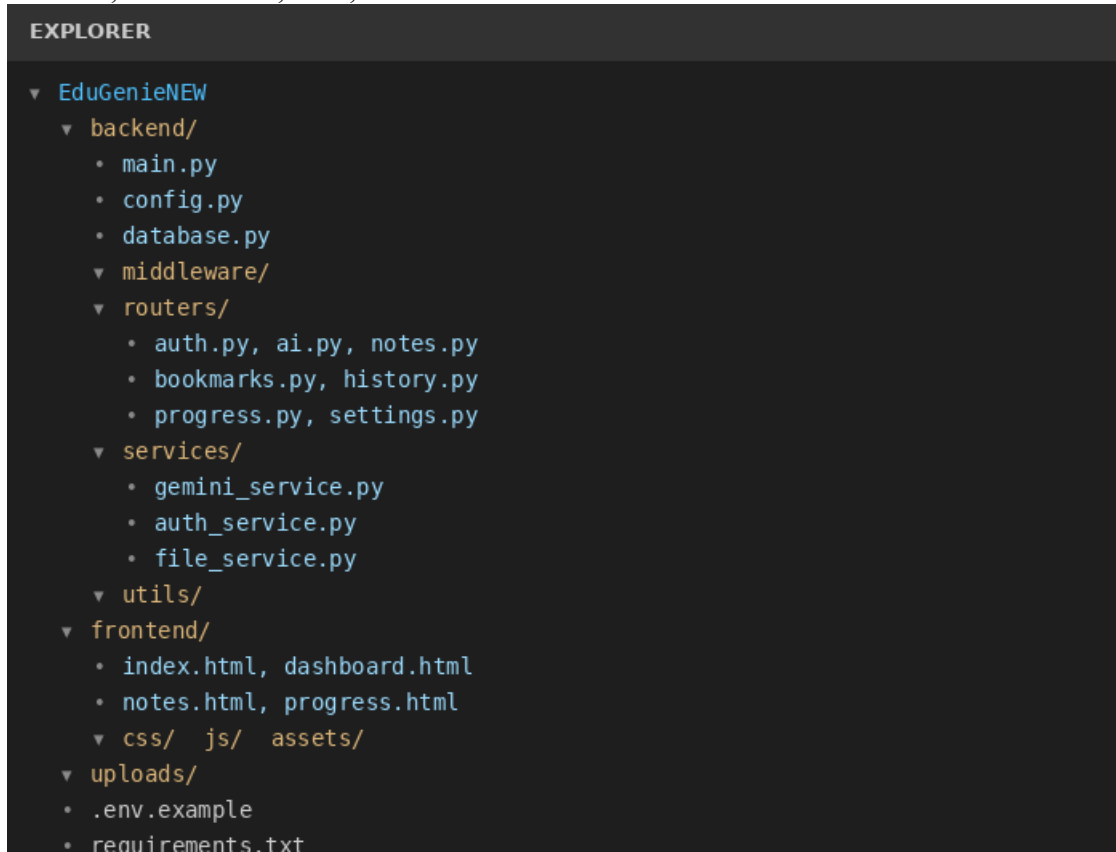


```

D:\projects>python -m venv myenv
D:\projects>"d:\projects\myenv\Scripts\activate"
(myenv) D:\projects> pip install -r requirements.txt
(myenv) D:\projects> pip install fastapi uvicorn motor
(myenv) D:\projects> pip install google-generativeai
(myenv) D:\projects> pip install python-jose passlib bcrypt
(myenv) D:\projects> pip install python-dotenv==1.0.1
  
```

Figure 3 — Installing project dependencies

- Configure the API Key: Create a .env file in the project root and add the Gemini API key and MongoDB connection string.
- Set Up Application Structure: Create the project directory structure including folders for routers, services, middleware, utils, and the frontend.



```

EXPLORER
└─ EduGenieNEW
   └─ backend/
      ├── main.py
      ├── config.py
      ├── database.py
      ├── middleware/
      ├── routers/
      │   ├── auth.py, ai.py, notes.py
      │   ├── bookmarks.py, history.py
      │   └── progress.py, settings.py
      ├── services/
      │   ├── gemini_service.py
      │   ├── auth_service.py
      │   └── file_service.py
      ├── utils/
      └─ frontend/
         ├── index.html, dashboard.html
         ├── notes.html, progress.html
         ├── css/
         ├── js/
         └─ assets/
      └─ uploads/
         ├── .env.example
         └─ requirements.txt
  
```


Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

Implement the core generation features — Question Answering, Concept Explanation, Quiz Generation, and Summarization — as unified AI responses driven by structured prompts.

- **Question Answering:** Generates detailed academic answers backed by curated context, with follow-up suggestions.
- **Concept Explanation:** Produces deep-dive explanations with interactive examples and analogies tailored to the student's level.
- **Quiz Generator:** Returns a structured set of multiple-choice questions with correct answers and difficulty ratings, calibrated to the requested topic.
- **Text Summarizer:** Condenses long-form text, PDF, or DOCX content into key takeaways in seconds.

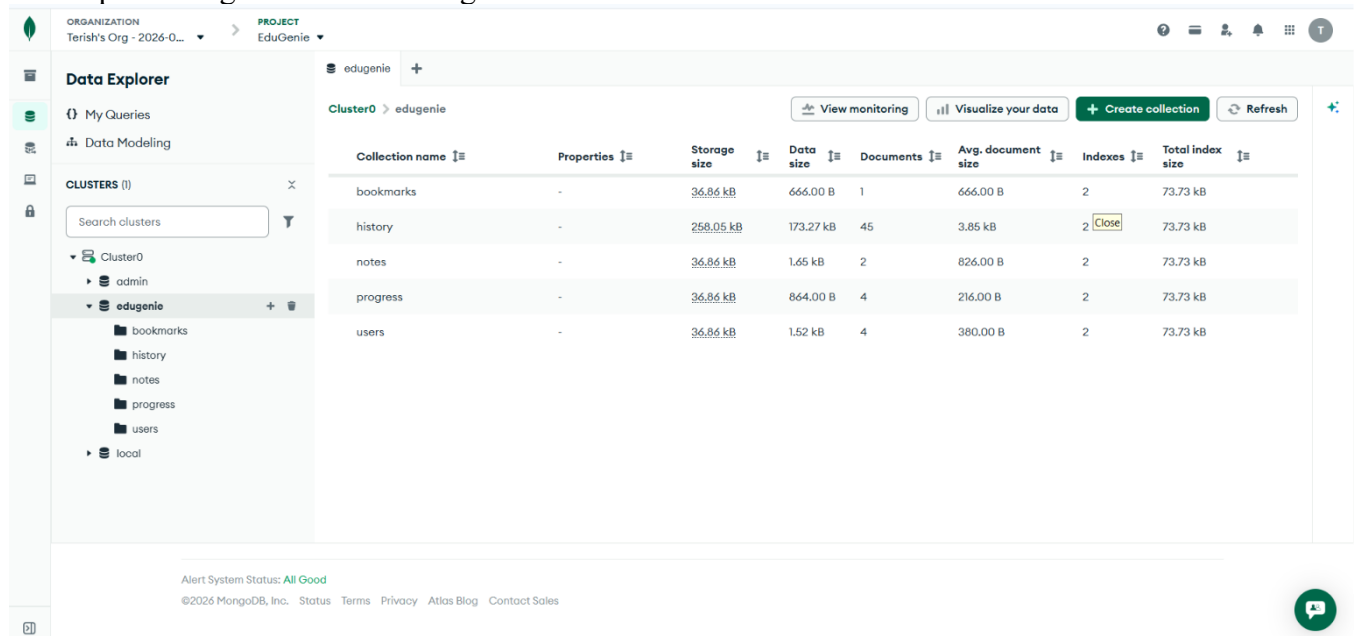
Activity 2.2: Implement the Fast API Backend

Define Routes:

- Set up routers in backend/routers/ for authentication, AI features, notes, bookmarks, history, progress, and settings.

Process User Input:

- Create Pydantic request models to validate incoming JSON payloads from the frontend.
- Use Fast Api's async request handling to retrieve JSON data submitted via fetch/AJAX calls.
- Validate that required fields such as question or document text are non-empty and within size limits before processing. Connect to MongoDB Atlas



The screenshot shows the MongoDB Atlas Data Explorer interface. On the left, the 'Data Explorer' sidebar shows a tree view of clusters. The 'edugenie' cluster is selected, showing a list of collections: bookmarks, history, notes, progress, and users. The main panel displays a table of these collections with their properties, storage size, data size, documents, average document size, indexes, and total index size.

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
bookmarks	-	36.86 kB	666.00 B	1	666.00 B	2	73.73 kB
history	-	258.05 kB	173.27 kB	45	3.85 kB	2	73.73 kB
notes	-	36.86 kB	1.65 kB	2	826.00 B	2	73.73 kB
progress	-	36.86 kB	864.00 B	4	216.00 B	2	73.73 kB
users	-	36.86 kB	1.52 kB	4	380.00 B	2	73.73 kB

Integrate API Calls:

- Within each AI route handler, call the `gemini_service.build_prompt()` helper to construct a structured prompt combining the feature type and user context.
- Submit the prompt to the Gemini API using the `google-generativeai` SDK's `generate_content()` method.
- Parse and validate the JSON response, handling direct JSON, markdown code fences, and raw text extraction as fallback strategies.

```
def extract_json(text: str) -> dict:
    """Extract JSON from Gemini response, handling markdown code blocks."""
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        pass
    match = re.search(r'```(?:json)?\s*([^\s\S]*)```', text)
    if match:
        try:
            return json.loads(match.group(1).strip())
        except json.JSONDecodeError:
            pass
    raise ValueError("Could not extract valid JSON from response")
```

Milestone 3: Backend (main.py) Development

Milestone 3 focuses on creating and refining the core logic of the Fast API application, which serves as the backbone of EduGenie. In this milestone, each feature's functionality is wired through routers, prompt construction and API interaction are made reliable, and each feature is tested end-to-end.

Activity 3.1: Writing the Main Application Logic

Define the Core Routes:

```
app = FastAPI(title="EduGenie API")

app.include_router(auth.router)
app.include_router(ai.router)
app.include_router(notes.router)
app.include_router(bookmarks.router)
app.include_router(history.router)
app.include_router(progress.router)
app.include_router(settings_router.router)
```

Example AI route handler:

```
@router.post("/qa")
async def question_answer(payload: QARequest, user=Depends(get_current_user)):
```

```

if not payload.question.strip():
    raise HTTPException(400, "Question is required.")
prompt = build_prompt("qa", payload.question, payload.context)
response = await gemini_service.generate(prompt)
result = extract_json(response)
await history_collection.insert_one({
    "user_id": user.id, "type": "qa", "result": result
})
return {"success": True, **result}

```

Define Prompt Engineering Helpers (utils/prompts.py):

FEATURE_INSTRUCTIONS dictionary: maps each AI feature to detailed instructions embedded in the prompt.

```

"""
EduGenie — Optimized Gemini Prompt Templates
Each function returns a formatted prompt string for the specific AI feature.
"""

def qa_prompt(question: str, context: str = "") -> str:
    ctx = f"\n\nAdditional context:\n{context}" if context else ""
    return f"""You are EduGenie, an expert AI educational assistant. Answer the following question
clearly, accurately, and educationally.

Use markdown formatting:
- Use bold for key terms
- Use code blocks for code snippets
- Use numbered lists or bullet points for steps
- Use tables when comparing items
- Use ## headings to organize long answers
- Use LaTeX notation  $...$  for math when needed

Always end with a brief "Key Takeaway" section.

Question: {question} {ctx}

Provide a thorough, well-structured answer that helps the student truly understand the concept."""

def explain_prompt(topic: str, level: str = "intermediate", style: str = "detailed") -> str:
    level_map = {
        "beginner": "a complete beginner (use simple language, analogies, and avoid jargon)",
        "intermediate": "an intermediate student (assume basic knowledge, go deeper)",
        "advanced": "an advanced student (use technical depth, edge cases, nuances)"
    }

```

```
style_map = {
    "simple": "Keep it simple, short and easy to understand.",
    "detailed": "Be thorough, cover all important aspects.",
    "examples": "Focus on practical examples and real-world applications.",
    "analogy": "Use real-world analogies and comparisons to explain.",
}
return f"""You are EduGenie, an expert AI educator. Explain the following topic for
{level_map.get(level, level_map['intermediate'])}.
```

Validate that the parsed result contains the expected keys before returning it to the frontend, and log any parsing failures for later review.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a clean, responsive interface for EduGenie. This involves designing a card-based dashboard layout with HTML and CSS, and creating dynamic content rendering with JavaScript to display AI-generated answers, quizzes, notes, and analytics.

Activity 4.1: Designing and Developing the User Interface

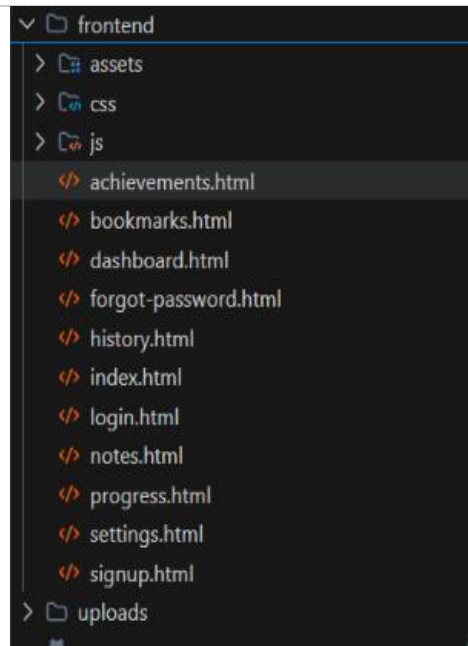
- Develop the main HTML pages (index.html, dashboard.html, notes.html, progress.html) as a multi-page responsive interface.
- Use semantic HTML elements to keep the structure organized and accessible.
- Integrate a clean card-based design system with a consistent blue accent colour, light and dark theme support, and Inter-style typography.

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet implementing a light, card-based aesthetic with soft shadows and rounded corners.
- Use flexbox and grid layouts to organize the dashboard into a sidebar, quick-action tools, and content panels.
- Add media queries to ensure the layout adapts across desktop, tablet, and mobile screen sizes.

Create Separate UI Components for Each Feature:

- Quick Action Cards: individual cards for Explain, Q&A, Quiz, Summarize, Notes, and Upload.
- Progress Widgets: study streak, focus score, sessions, and weekly goal cards.
- Quiz Cards: multiple-choice question cards with instant feedback.



Activity 4.2: Creating Dynamic Content Rendering with JavaScript

- Attach a submit event listener to each generator form that prevents default submission and sends data as a JSON POST request via the Fetch API.
- During loading, disable the submit button and show an animated loader in place of the button text.
- Implement a `renderResults(data)` function that dynamically creates and injects DOM elements for answers, explanations, and quiz questions into their respective containers.
- After rendering, automatically smooth-scroll the viewport to the results section using `scrollIntoView()`.
- In the finally block of the async handler, re-enable the submit button and restore its original state regardless of success or failure.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying EduGenie first locally to verify correct operation, and then to a cloud platform to share the application over a secure, accessible URL. This involves configuring the server environment, managing dependencies, and launching the app for real-world use.

Activity 5.1: Preparing the Application for Local Deployment

- Create and activate a virtual environment, then install all required dependencies from `requirements.txt`.
- Configure environment variables: create a `.env` file in the project root with `GEMINI_API_KEY`, `MONGODB_URI`, and `JWT_SECRET_KEY`.

Activity 5.2: Local Testing and Verification

- Run the application locally using `uvicorn`.

```
PS C:\Users\I Terish charan tej\Downloads\EduGenieNEW\5. Project Development Phase> uvicorn backend.main:app
p --reload --host 0.0.0.0 --port 8000
INFO: Will watch for changes in these directories: ['C:\Users\I Terish charan tej\Downloads\EduGenieNEW\5. Project Development Phase']
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Started reloader process [22864] using WatchFiles
INFO: Started server process [34264]
INFO: Waiting for application startup.
2026-07-03 15:58:30 | INFO | backend.main | 🚀 EduGenie starting up...
2026-07-03 15:58:31 | INFO | pymongo.serverSelection | {"message": "waiting for suitable server to become a
available", "selector": "Primary()", "operation": "ping", "topologyDescription": "<TopologyDescription id: 6
a478ece561cd0e4116bf129, topology_type: ReplicaSetNoPrimary, servers: [<ServerDescription ('ac-ttyffl4-shar
d-00-00.hicacuz.mongodb.net', 27017) server_type: Unknown, rtt: None>, <ServerDescription ('ac-ttyffl4-shar
d-00-01.hicacuz.mongodb.net', 27017) server_type: Unknown, rtt: None>, <ServerDescription ('ac-ttyffl4-shar
d-00-02.hicacuz.mongodb.net', 27017) server_type: Unknown, rtt: None>]", "clientId": {"$oid": "6a478ece561
cd0e4116bf129"}, "remainingTimeMS": 30}
2026-07-03 15:58:32 | INFO | backend.database | ✅ Connected to MongoDB: edugenie
2026-07-03 15:58:32 | INFO | backend.database | Database indexes created
INFO: Application startup complete.
PS C:\Users\I Terish charan tej\Downloads\EduGenieNEW\5. Project Development Phase>
```

- Once running, open a browser and navigate to <http://localhost:8000> to access the app.
- Interact with each AI tool — Q&A, Explain, Quiz, Summarize, and Roadmap — to verify responses are correctly generated, parsed, and rendered.

Activity 5.3: Public Deployment

EduGenie can be deployed to a managed cloud host such as Render, Railway, or Fly.io, removing the need to manage server infrastructure directly.

- Push the repository to GitHub and connect it to the chosen hosting provider.
- Configure the build command (`pip install -r requirements.txt`) and start command (`uvicorn backend.main:app --host 0.0.0.0 --port $PORT`).
- Add the environment variables (`GEMINI_API_KEY`, `MONGODB_URI`, `JWT_SECRET_KEY`) in the hosting provider's dashboard.
- Deploy the frontend static assets to Vercel or Netlify, pointing API calls to the deployed backend URL.

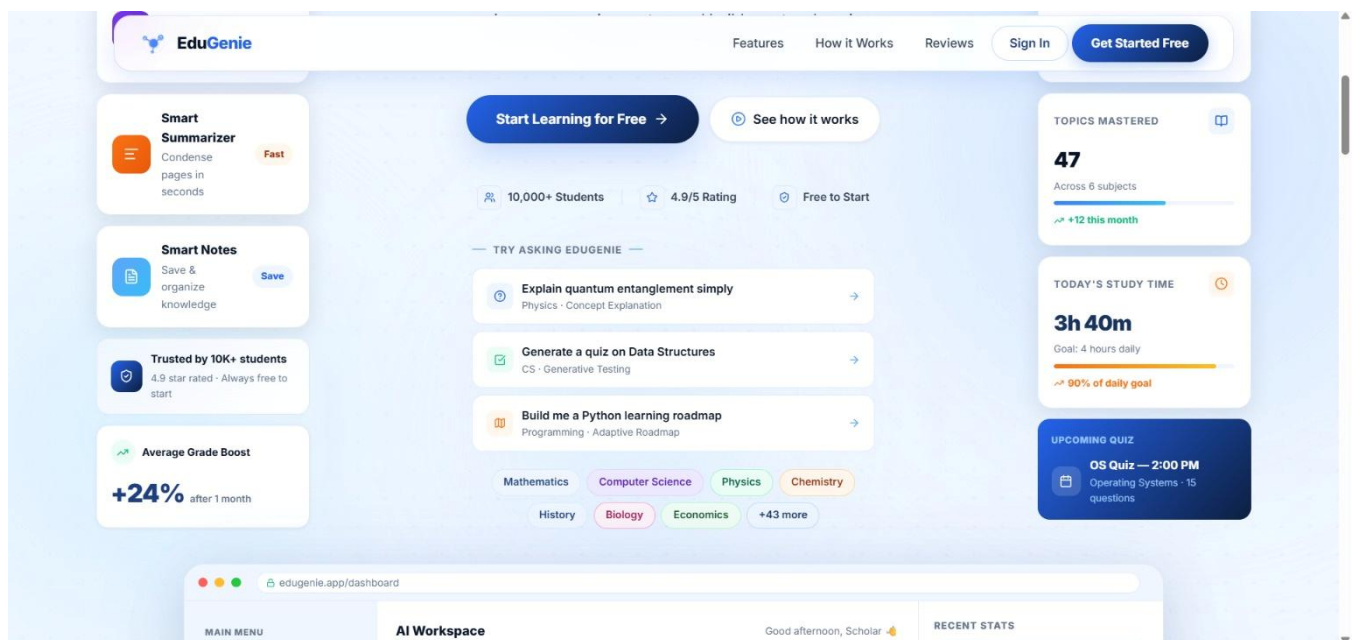
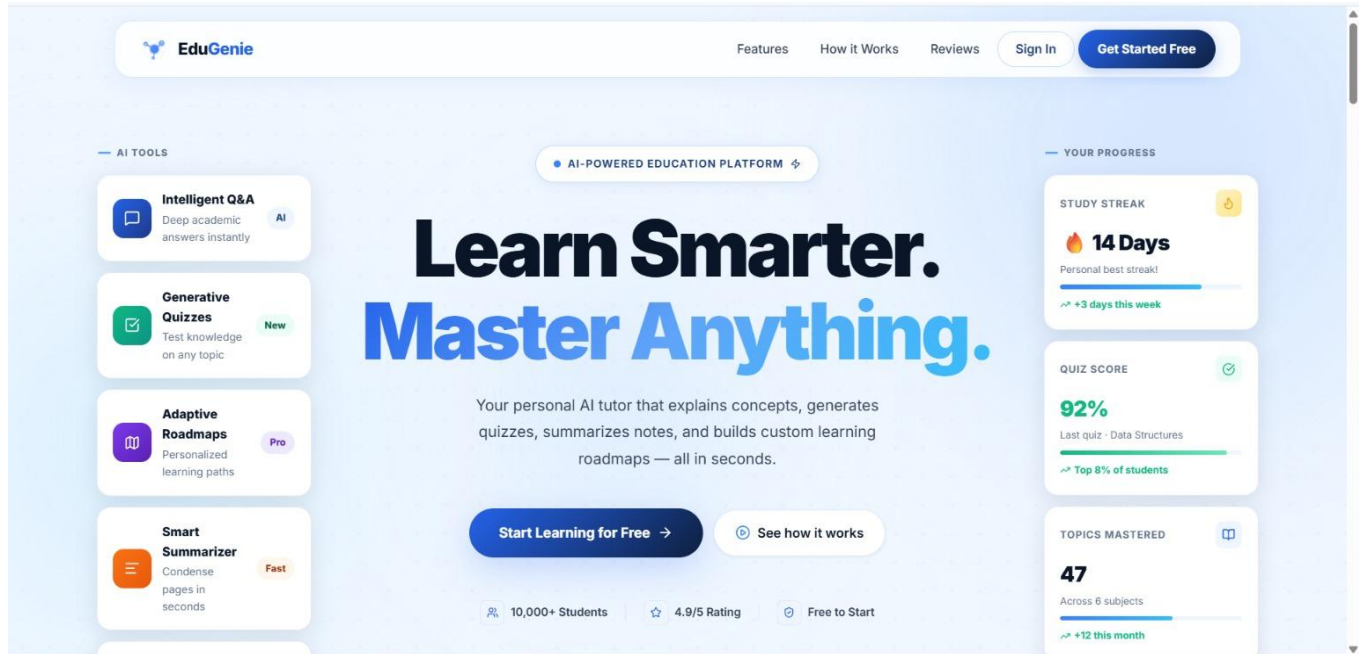
Important Notes:

- `DEBUG=False` is recommended in production to avoid exposing detailed error traces.
- Use a paid MongoDB Atlas tier or configure IP allowlisting for production traffic.
- Rotate the JWT secret key periodically and store all secrets using the hosting provider's secret manager.

Exploring the Website's Web Pages:

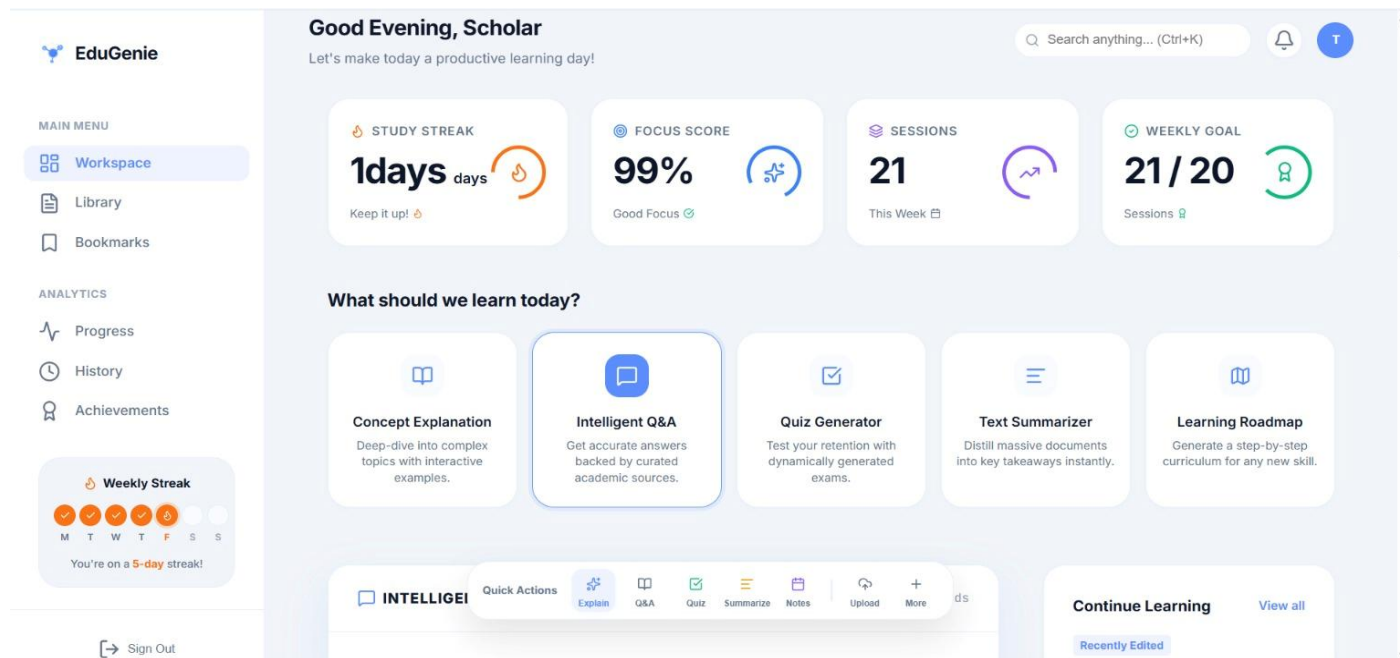
Landing Page:

The landing page introduces EduGenie's AI tools and value proposition. It features a hero section with a clear call to action, a sidebar preview of the available AI tools, and a live snapshot of the student's progress, including study streak, quiz score, and topics mastered.



Dashboard / Workspace Page:

The workspace dashboard is the student's home base after signing in. It surfaces daily study statistics — streak, focus score, sessions, and weekly goal — alongside quick-action cards for each AI tool (Concept Explanation, Intelligent Q&A, Quiz Generator, Text Summarizer, and Learning Roadmap) and a panel for continuing recently edited work.



EduGenie

Good Evening, Scholar
Let's make today a productive learning day!

Search anything... (Ctrl+K)

STUDY STREAK
1days days
Keep it up!

FOCUS SCORE
99%
Good Focus

SESSIONS
21
This Week

WEEKLY GOAL
21/20
Sessions

What should we learn today?

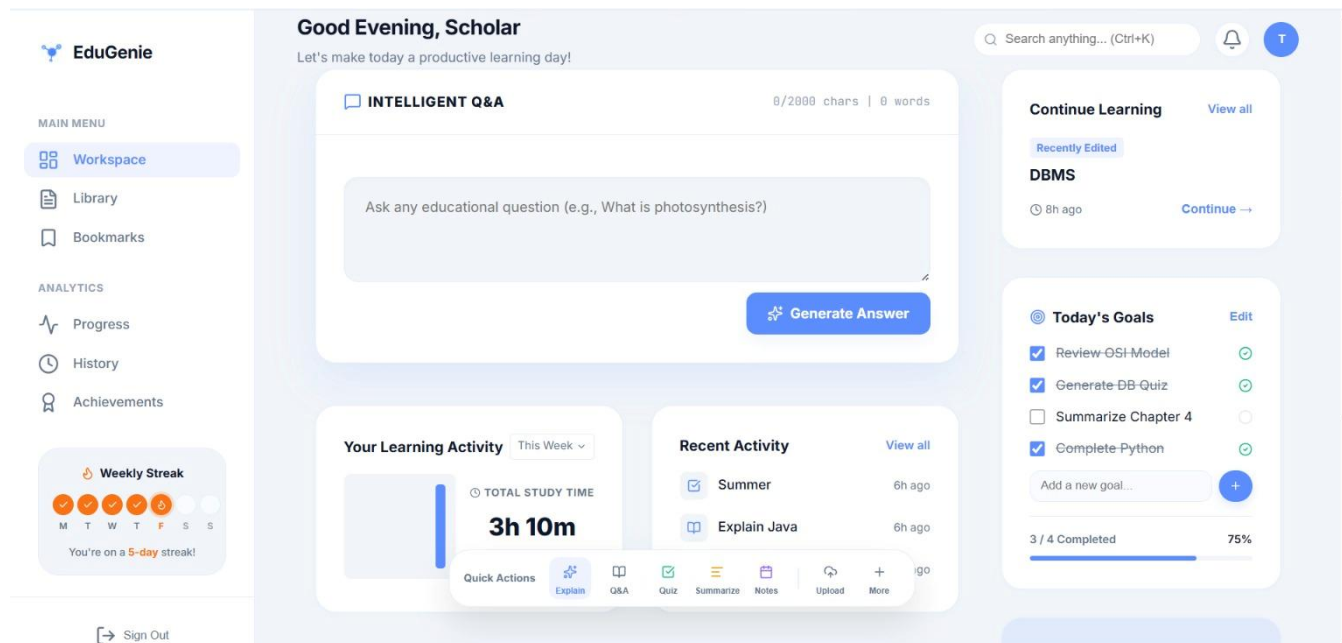
- Concept Explanation**
Deep-dive into complex topics with interactive examples.
- Intelligent Q&A**
Get accurate answers backed by curated academic sources.
- Quiz Generator**
Test your retention with dynamically generated exams.
- Text Summarizer**
Distill massive documents into key takeaways instantly.
- Learning Roadmap**
Generate a step-by-step curriculum for any new skill.

Weekly Streak
M T W T F S S
You're on a 5-day streak!

INTELLIGIE Quick Actions: Explain, Q&A, Quiz, Summarize, Notes, Upload, More

Continue Learning View all
Recently Edited

Sign Out



EduGenie

Good Evening, Scholar
Let's make today a productive learning day!

Search anything... (Ctrl+K)

INTELLIGIE 0/2000 chars | 0 words

Ask any educational question (e.g., What is photosynthesis?)

Generate Answer

Continue Learning View all
Recently Edited
DBMS
8h ago Continue →

Today's Goals Edit

- ☒ Review OSI Model
- ☒ Generate DB Quiz
- ☐ Summarize Chapter 4
- ☒ Complete Python

Add a new goal...

3 / 4 Completed 75%

Your Learning Activity This Week

Recent Activity View all

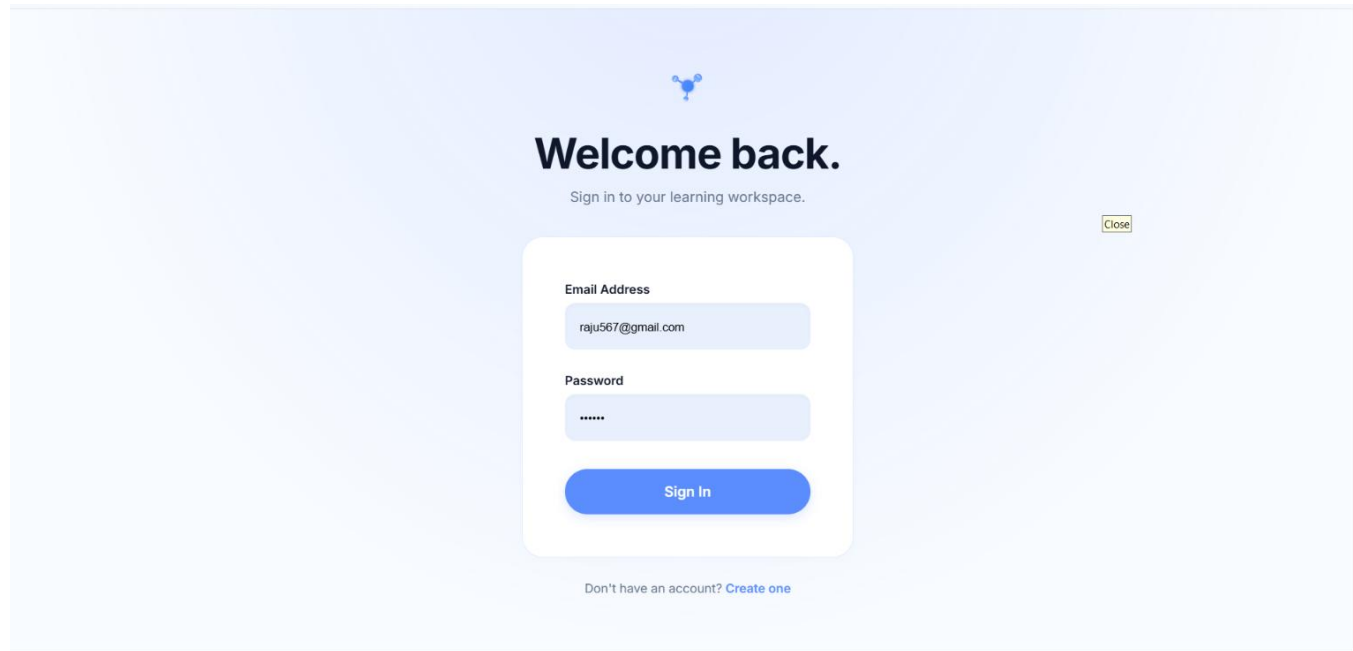
- Summer 6h ago
- Explain Java 6h ago

Quick Actions Explain, Q&A, Quiz, Summarize, Notes, Upload, More

Sign Out

Login Page:

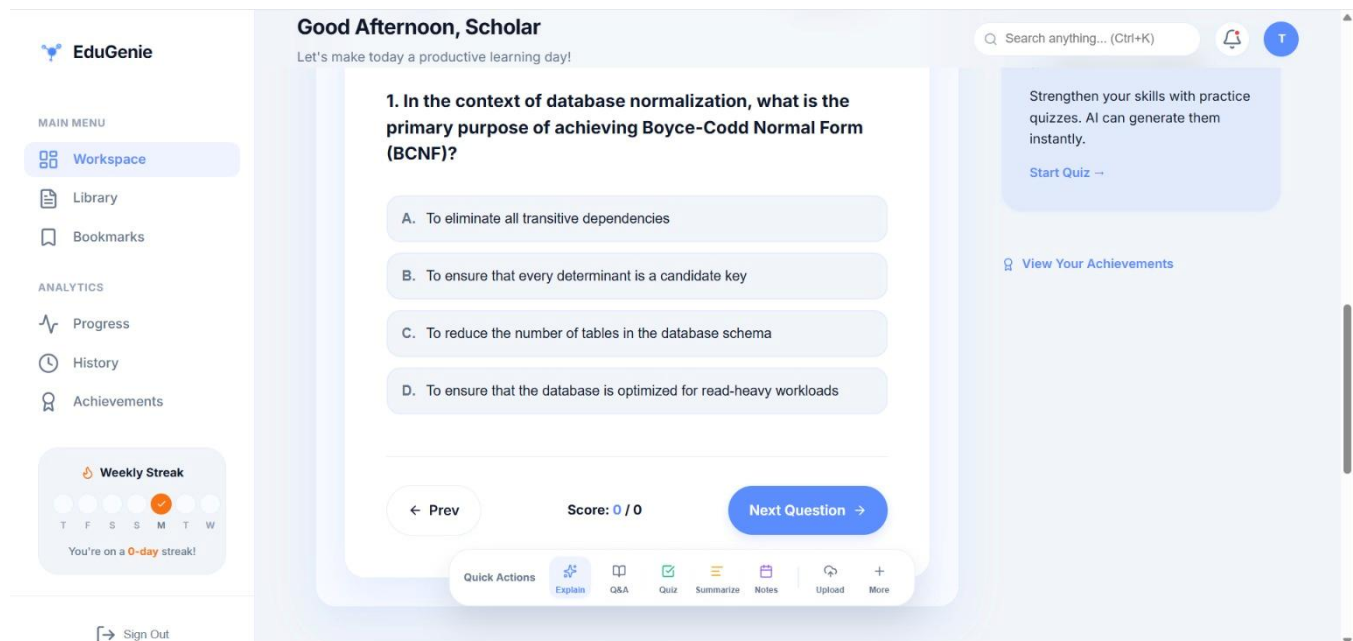
A minimal, distraction-free sign-in screen consistent with the dashboard's visual language, offering email/password authentication and a link to the sign-up flow.



The login page features a light blue background with a central white card for authentication. At the top of the card, the text "Welcome back." is displayed in a large, bold font, followed by the instruction "Sign in to your learning workspace." Below this, there are two input fields: "Email Address" containing "raju567@gmail.com" and "Password" with masked characters. A blue "Sign In" button is positioned below the password field. At the bottom of the card, a link "Don't have an account? Create one" is provided. A small "Close" button is located in the top right corner of the page.

Quiz Generator Module:

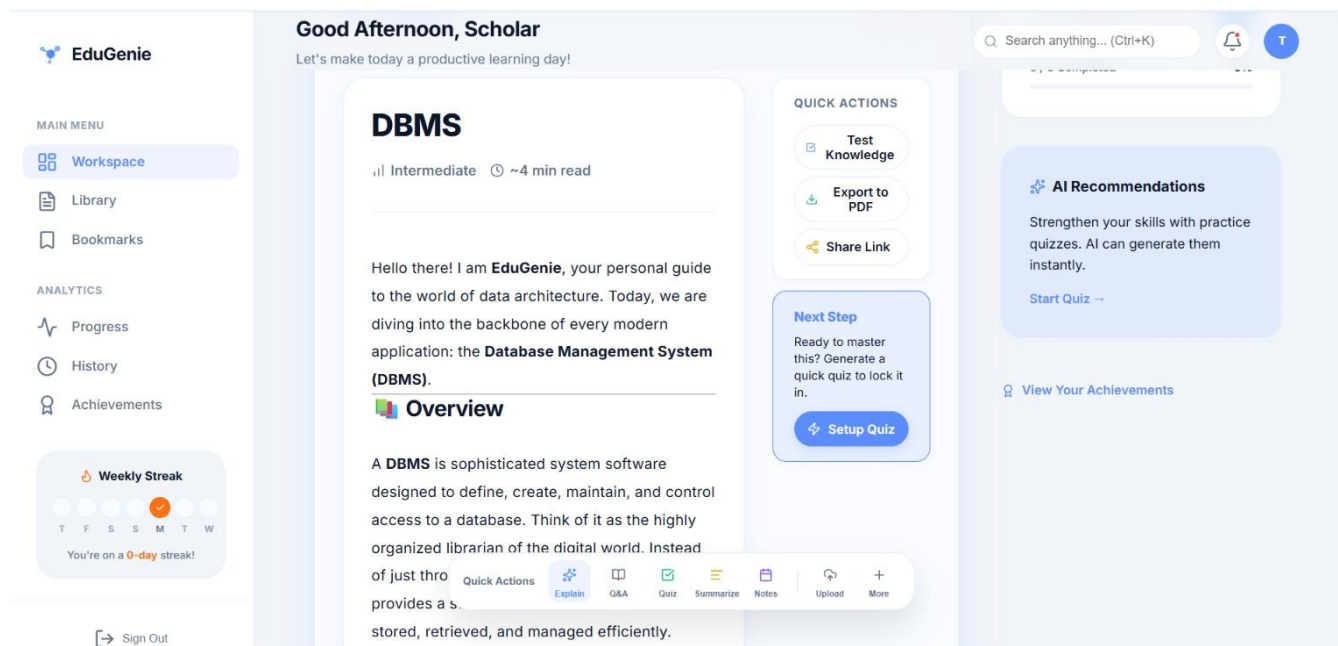
Students choose a topic and instantly receive a set of AI-generated multiple-choice questions, each tagged with difficulty and immediate feedback on selection.



The screenshot shows the "Good Afternoon, Scholar" interface. The main content area displays a question: "1. In the context of database normalization, what is the primary purpose of achieving Boyce-Codd Normal Form (BCNF)?" with four multiple-choice options: A. To eliminate all transitive dependencies, B. To ensure that every determinant is a candidate key, C. To reduce the number of tables in the database schema, and D. To ensure that the database is optimized for read-heavy workloads. Below the options are "Prev" and "Next Question" buttons, along with a "Score: 0 / 0" indicator. The left sidebar contains a "MAIN MENU" with "Workspace", "Library", and "Bookmarks", and an "ANALYTICS" section with "Progress", "History", and "Achievements". A "Weekly Streak" widget shows a 0-day streak. The bottom navigation bar includes "Quick Actions" like "Explain", "Q&A", "Quiz", "Summarize", "Notes", "Upload", and "More". The right sidebar features a search bar, a notification bell, a user profile icon, and a "View Your Achievements" link.

Concept Explanation Module:

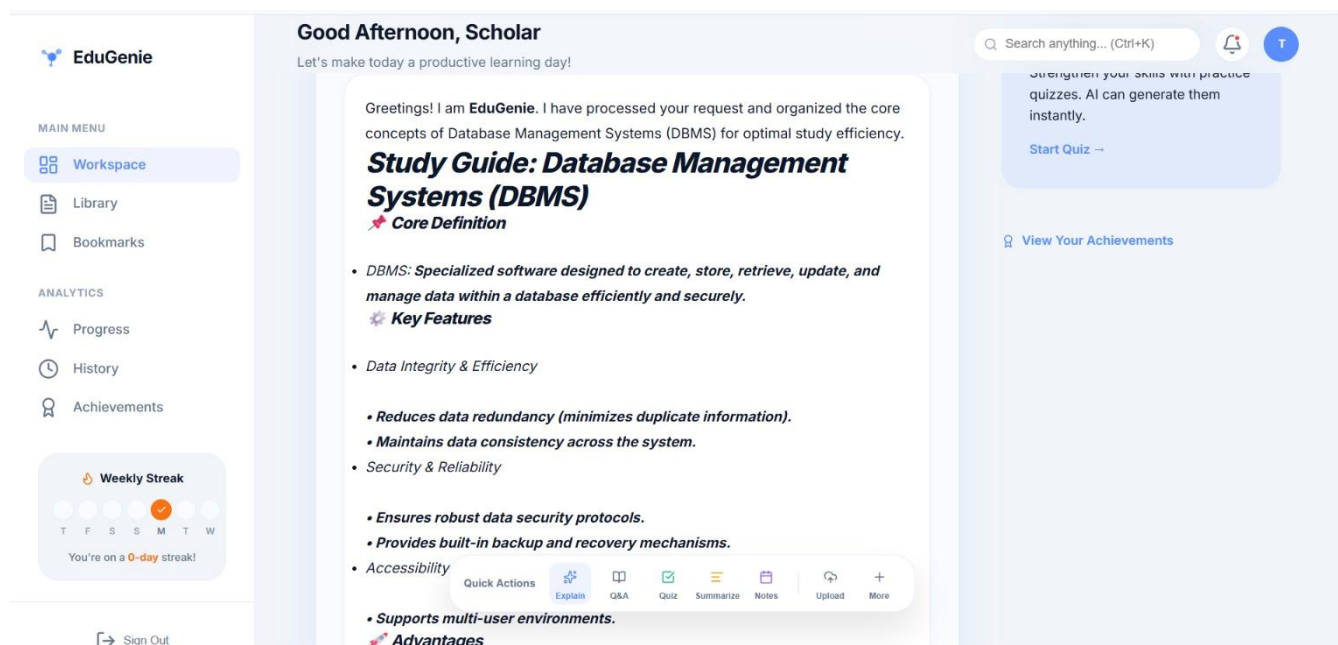
The Concept Explanation module provides detailed and structured explanations of academic topics using Google Gemini AI. It presents concepts with definitions, key functions, examples, and organized sections to help learners understand complex subjects more effectively.



The screenshot shows the EduGenie interface for the Concept Explanation Module. The main content area displays a detailed explanation of Database Management Systems (DBMS). The interface includes a sidebar with navigation options like Workspace, Library, and Bookmarks, and a top navigation bar with a search bar and user profile. The main content area is titled "Good Afternoon, Scholar" and "Let's make today a productive learning day!". The topic "DBMS" is highlighted, and the text explains its role in data architecture. A "Weekly Streak" section shows a 0-day streak. The interface also features a "QUICK ACTIONS" panel with options like Test Knowledge, Export to PDF, and Share Link, and a "Next Step" section with a "Setup Quiz" button.

Text Summarizer Module:

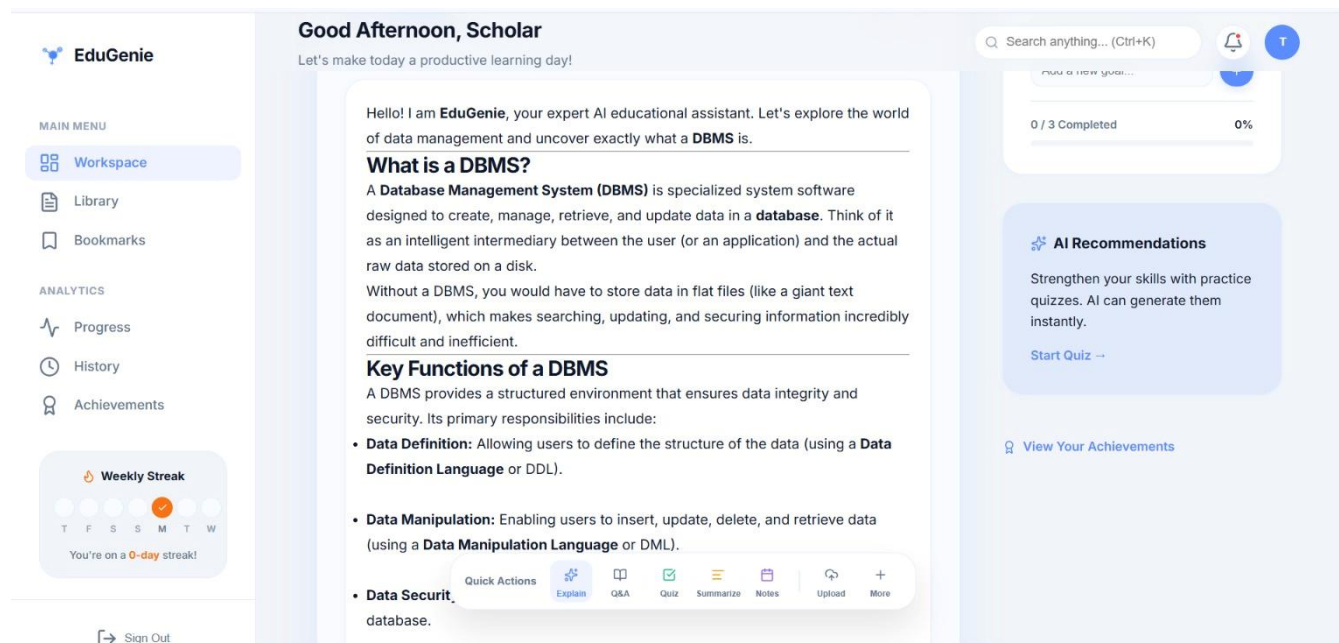
The Text Summarizer module converts long educational content into concise and meaningful summaries. It highlights key points to support quick revision and efficient learning.



The screenshot shows the EduGenie interface for the Text Summarizer Module. The main content area displays a summarized version of the DBMS concept. The interface includes a sidebar with navigation options like Workspace, Library, and Bookmarks, and a top navigation bar with a search bar and user profile. The main content area is titled "Good Afternoon, Scholar" and "Let's make today a productive learning day!". The topic "DBMS" is highlighted, and the text provides a concise summary of its core definition and key features. A "Weekly Streak" section shows a 0-day streak. The interface also features a "QUICK ACTIONS" panel with options like Test Knowledge, Export to PDF, and Share Link, and a "Next Step" section with a "Setup Quiz" button.

Intelligent Q&A Module:

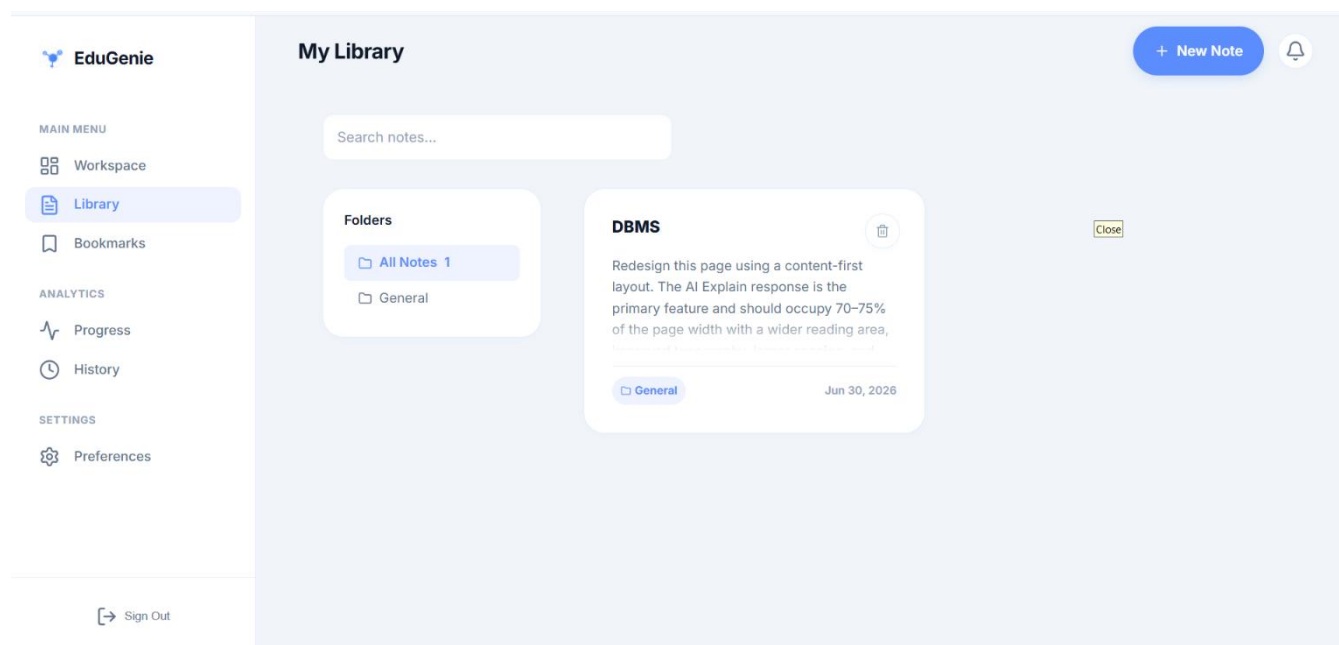
The Intelligent Q&A module allows users to ask academic questions and receive accurate, context-aware answers generated by AI. It is designed to support quick doubt-solving and exam preparation.



The screenshot shows the EduGenie interface with a sidebar menu on the left containing 'Workspace', 'Library', 'Bookmarks', 'Progress', 'History', and 'Achievements'. The main content area is titled 'Good Afternoon, Scholar' and features a chat window with the AI assistant EduGenie. The chat text includes a greeting, a definition of a DBMS, and a list of its functions: Data Definition, Data Manipulation, and Data Security. On the right, there is a progress bar showing '0 / 3 Completed' and '0%', along with an 'AI Recommendations' section suggesting practice quizzes. A 'Weekly Streak' widget shows a 0-day streak. At the bottom, there is a 'Sign Out' button.

Smart Notes Page:

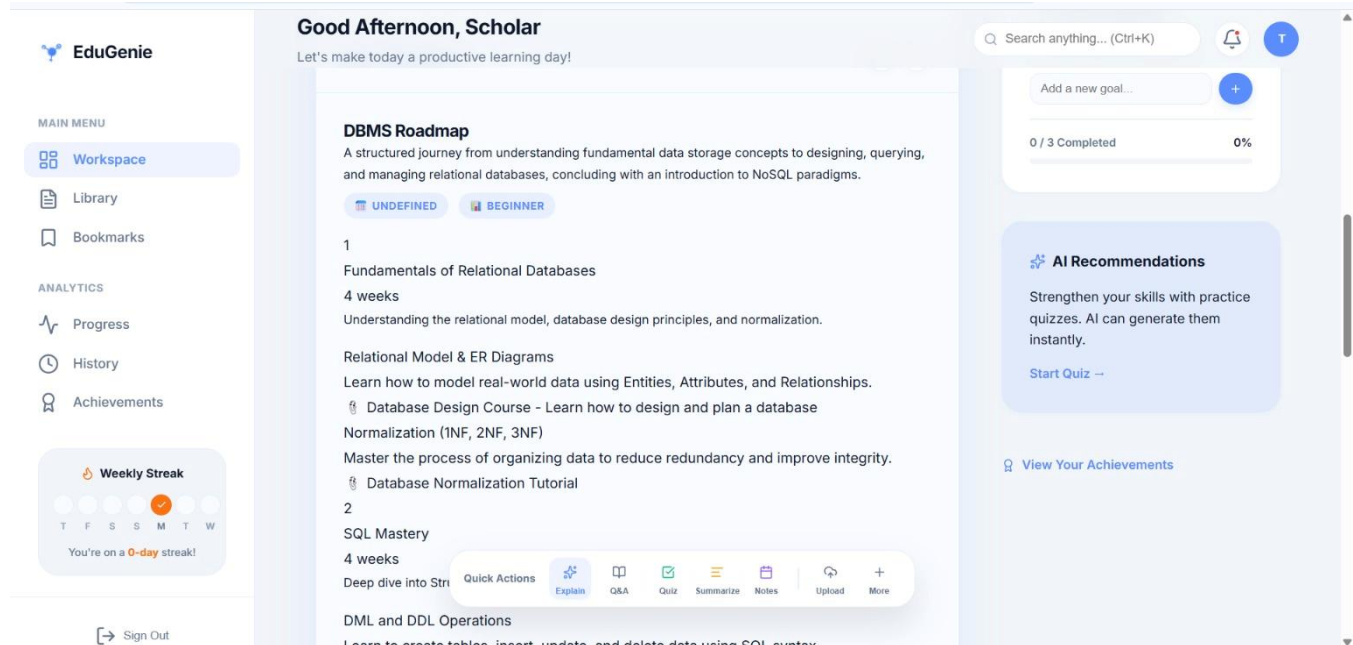
Notes are organized into folders, autosaved as the student types, and searchable across all subjects — reducing friction between learning and note-taking.



The screenshot shows the 'My Library' section of the EduGenie interface. The sidebar menu on the left is the same as in the previous screenshot, but 'Library' is highlighted. The main content area has a search bar labeled 'Search notes...'. Below the search bar, there is a 'Folders' section with 'All Notes 1' and 'General'. A note titled 'DBMS' is displayed, containing text about redesigning the page layout. The note is dated 'Jun 30, 2026' and has a 'General' tag. A 'New Note' button and a bell icon are in the top right corner. A 'Close' button is visible near the note.

Learning Roadmap Module:

The Learning Roadmap module generates a structured step-by-step learning plan for any topic or skill. It guides learners from beginner to advanced levels in a systematic progression.



Good Afternoon, Scholar
 Let's make today a productive learning day!

DBMS Roadmap
 A structured journey from understanding fundamental data storage concepts to designing, querying, and managing relational databases, concluding with an introduction to NoSQL paradigms.

1
Fundamentals of Relational Databases
 4 weeks
 Understanding the relational model, database design principles, and normalization.

Relational Model & ER Diagrams
 Learn how to model real-world data using Entities, Attributes, and Relationships.

Database Design Course - Learn how to design and plan a database

Normalization (1NF, 2NF, 3NF)
 Master the process of organizing data to reduce redundancy and improve integrity.

Database Normalization Tutorial

2
SQL Mastery
 4 weeks
 Deep dive into Stri

DML and DDL Operations
 Learn to create tables, insert, update, and delete data using SQL syntax.

Quick Actions: Explain, Q&A, Quiz, Summarize, Notes, Upload, More

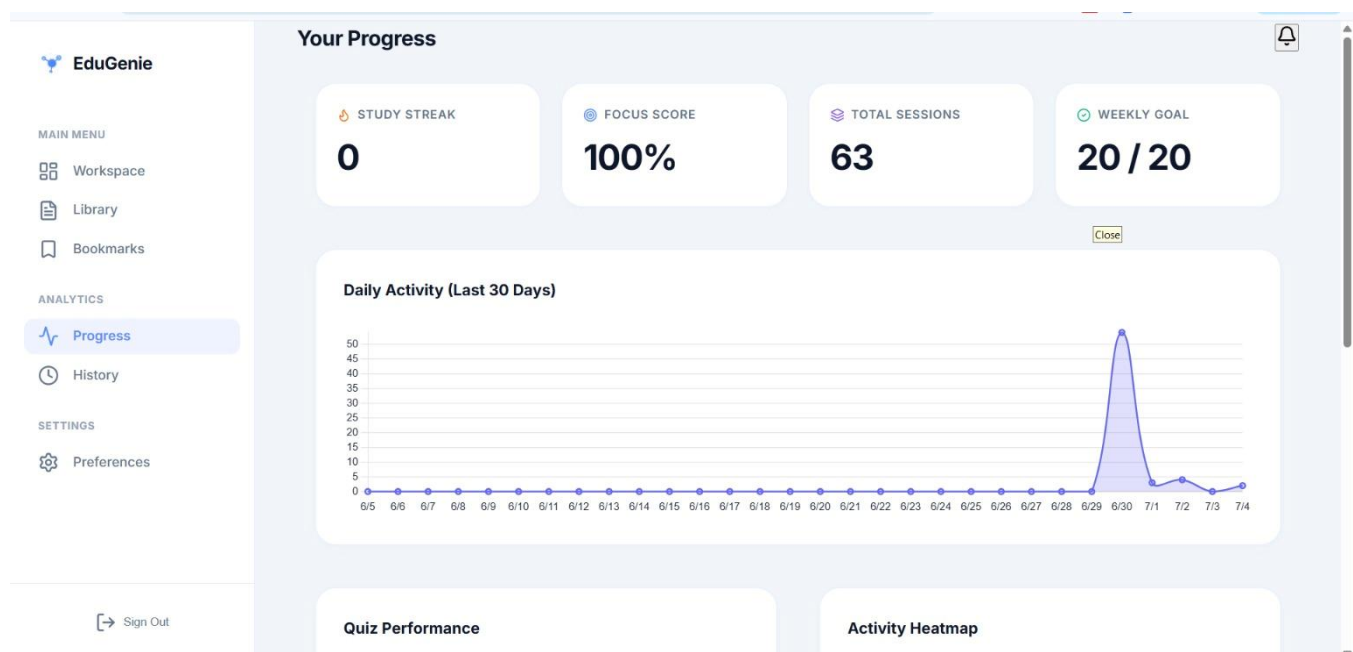
Weekly Streak: You're on a 0-day streak!

AI Recommendations: Strengthen your skills with practice quizzes. AI can generate them instantly. [Start Quiz](#)

[View Your Achievements](#)

Progress / Analytics Page:

The analytics page visualizes learning consistency and outcomes over time, giving students a clear picture of their study streak, quiz performance, and weekly activity trends.



Your Progress

STUDY STREAK: 0

FOCUS SCORE: 100%

TOTAL SESSIONS: 63

WEEKLY GOAL: 20 / 20

Daily Activity (Last 30 Days)

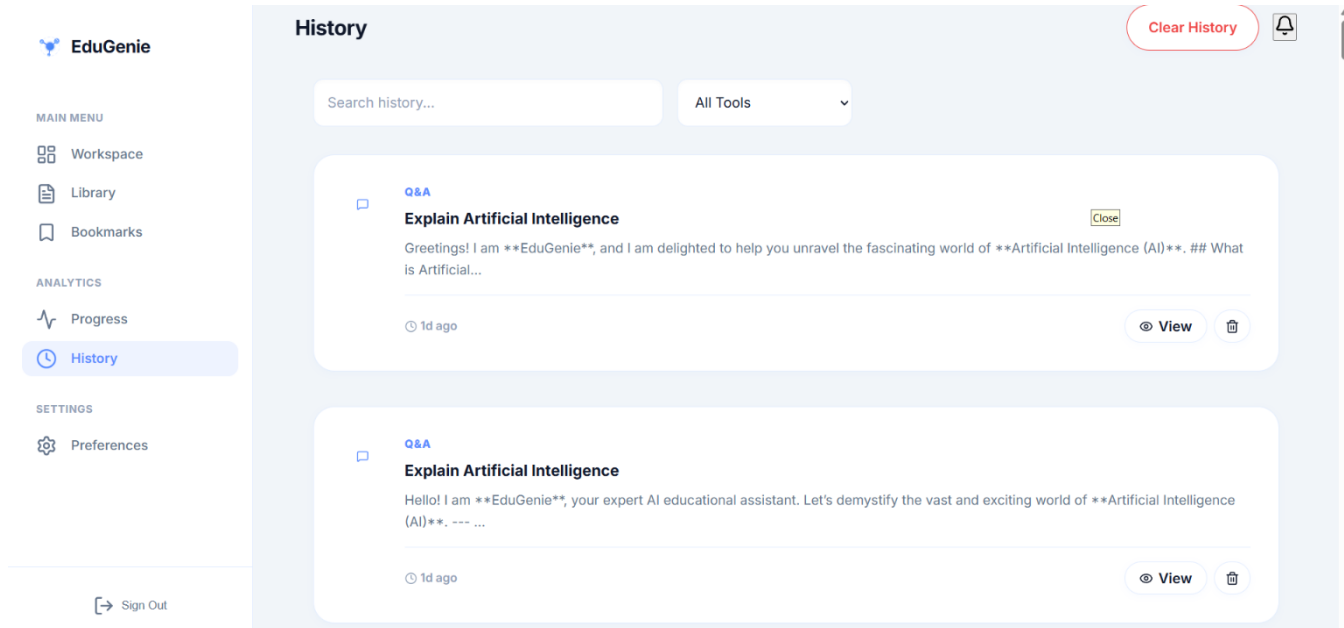
Line graph showing activity levels over time. The x-axis represents dates from 6/5 to 7/4. The y-axis represents activity level from 0 to 50. A significant peak is visible around 6/30, reaching approximately 45.

Quiz Performance

Activity Heatmap

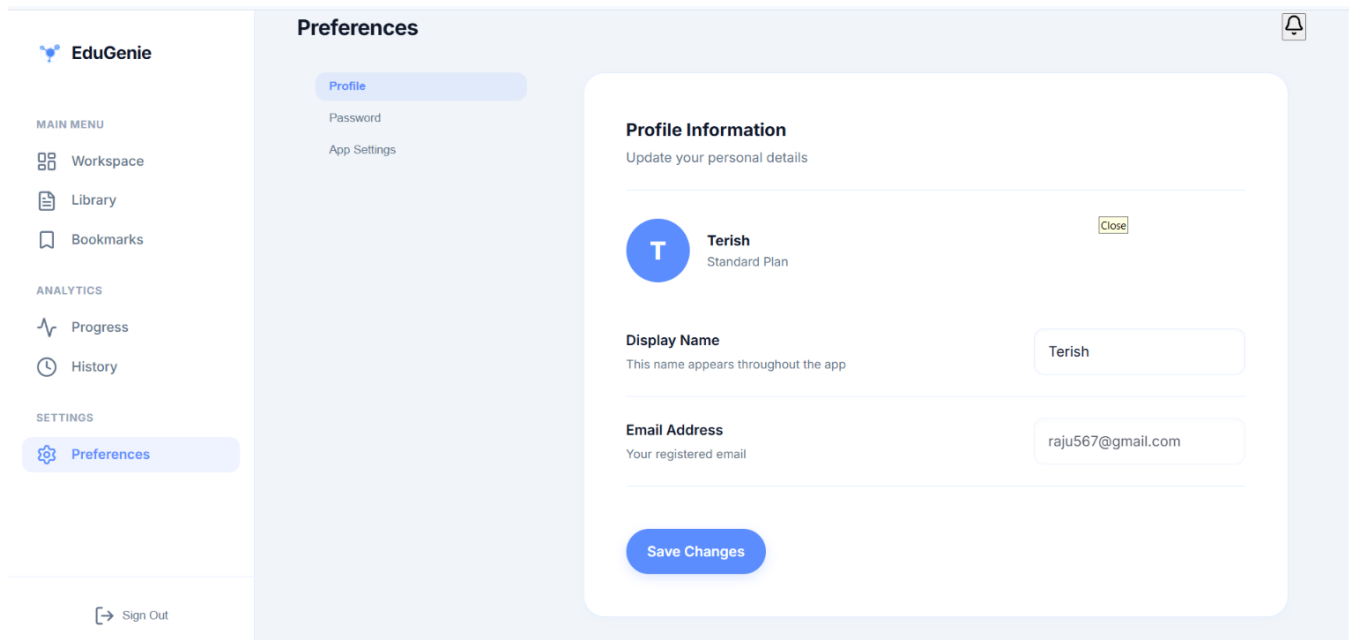
History Management Interface:

The History Management Interface stores and organizes all previous AI interactions, allowing users to search, filter, review, or delete past responses. This feature enables learners to revisit previously generated content without repeating the same queries.



User Preferences Interface:

The User Preferences Interface allows users to manage their personal profile, account settings, password, and application preferences. It provides a centralized location for updating user information and customizing the EduGenie experience according to individual needs.



Milestone 6: Project Conclusion and Future Enhancements

The final milestone of the EduGenie project focuses on ensuring the successful completion, validation, and documentation of the entire system. At this stage, all developed modules are thoroughly verified to confirm that the application meets its intended objectives of providing an intelligent, reliable, and interactive AI-based educational platform. The system is evaluated for functional correctness, integration stability, and overall performance to ensure a smooth user experience across all features.

Along with system validation, comprehensive documentation is prepared covering the complete development lifecycle, including system architecture, API design, backend implementation, frontend structure, database schema, and testing methodology. This documentation serves as a reference for academic evaluation as well as future improvements and maintenance of the platform.

Finally, potential enhancements are identified to extend the capabilities of EduGenie beyond its current version. These improvements aim to make the platform more intelligent, scalable, and user-friendly by incorporating advanced AI features, improved learning personalization, and cross-platform accessibility.

Activity 6.1: Final System Validation and Testing

In this phase, the complete EduGenie application is tested to ensure that all modules function correctly and work together seamlessly. Functional testing is performed across all major components, including authentication, AI-powered question answering, concept explanation, quiz generation, text summarization, learning roadmap generation, notes management, bookmarks, progress tracking, history, and settings.

Additionally, performance testing is conducted using Apache JMeter on key API endpoints such as `/api/auth/login` and `/api/ai/qa` to evaluate system behavior under different load conditions. Metrics such as response time, throughput, and error handling are analyzed to ensure system reliability and stability.

Activity 6.2: Documentation and Project Completion

The complete development process is documented in detail to support academic submission and future reference. This includes system architecture diagrams, API endpoint documentation, database design structure, implementation details, and testing results.

All project artifacts such as source code, configuration files, environment setup instructions, and user manuals are systematically organized to ensure clarity and maintainability. This documentation also supports future enhancements and scalability of the system.

Activity 6.3: Future Enhancements

To further improve the capabilities of EduGenie, several future enhancements are proposed. These include the integration of voice-based AI tutoring for interactive learning, AI-powered flashcards using spaced repetition techniques, and multilingual support to reach a wider audience. Additionally, collaborative learning features can be introduced to enable group study environments and peer interaction.

Further improvements may also include the development of dedicated mobile applications for Android and iOS, integration of advanced caching mechanisms such as Redis for performance optimization, offline learning capabilities, and improved personalization using adaptive AI models. Scalability enhancements such as cloud deployment with auto-scaling and load balancing can also be incorporated to support a larger user base.